

1. Gray Scale Frame Matrix

Gray Scale Frame Matrix

[2]
✓ 0s

```
import cv2
import numpy as np

# Load the video
video_path = "sample_video.mp4" # change to your video file path
cap = cv2.VideoCapture(video_path)

# Read one frame
ret, frame = cap.read()

if ret:
    # Convert frame to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Print grayscale matrix values
    print("Grayscale Frame Matrix:\n")
    print(gray)

else:
    print("Error: Could not read frame from video.")

# Release video
cap.release()
```

▼

Grayscale Frame Matrix:

```
[[0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]]
```

2. Video Capturing

```
[ ] import cv2

cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Camera not detected")
    exit()

ret, frame = cap.read()

if not ret:
    print("Failed to capture image")
    exit()

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
negative = 255 - gray

print("Original Matrix:\n", gray)
print("\nNegative Matrix:\n", negative)

cv2.imshow("Original", gray)
cv2.imshow("Negative", negative)

cv2.waitKey(0)
cv2.destroyAllWindows()

cap.release()
```

Camera not detected
Failed to capture image

error Traceback (most recent call last)
/tmp/ipykernel_366/1390013637.py in <cell line: 0>()
13 exit()
14
----> 15 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
16 negative = 255 - gray
17
error: OpenCV(4.13.0) /io/opencv/modules/imgproc/src/color.cpp:199: error: (-215:Assertion failed) !_src.empty() in function 'cvtColor'

Problem: In colab python notebook cannot directly able to access web camera

```
from IPython.display import Javascript, display
from google.colab.output import eval_js
from base64 import b64decode

def record_video(filename='video.webm'):

    js = Javascript('''
    async function recordVideo() {
        const div = document.createElement('div');
        const start = document.createElement('button');
        const stop = document.createElement('button');

        start.textContent = "Start Recording";
        stop.textContent = "Stop Recording";

        div.appendChild(start);
        div.appendChild(stop);

        const video = document.createElement('video');
        video.style.display = 'block';
        div.appendChild(video);

        document.body.appendChild(div);

        const stream = await navigator.mediaDevices.getUserMedia({video: true});
        video.srcObject = stream;
        await video.play();

        let recorder = new MediaRecorder(stream);
        let chunks = [];

        recorder.ondataavailable = e => chunks.push(e.data);

        start.onclick = () => recorder.start();

        stop.onclick = () => recorder.stop();

        await new Promise(resolve => recorder.onstop = resolve);

        const blob = new Blob(chunks);
        const reader = new FileReader();

        return await new Promise(resolve => {
            reader.onloadend = () => resolve(reader.result);
            reader.readAsDataURL(blob);
        });
    }
    ''')

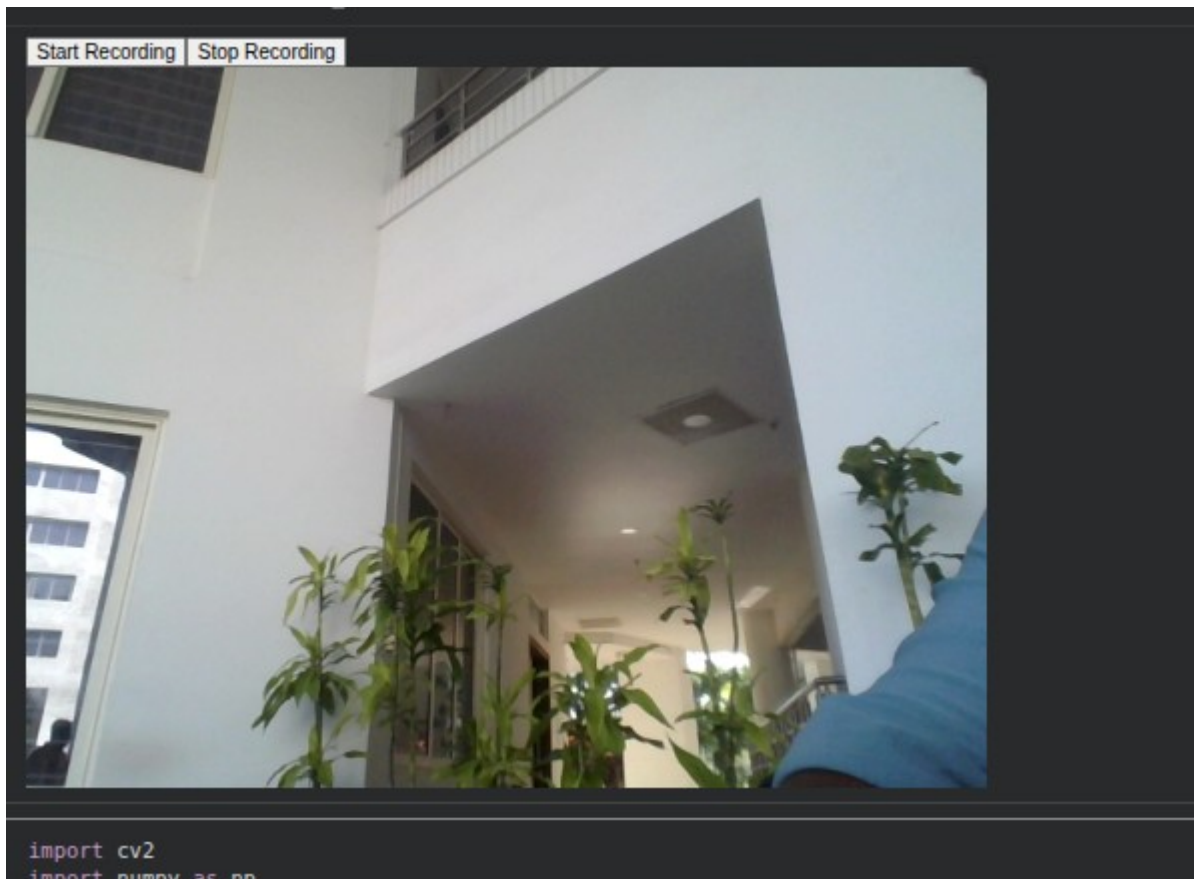
    display(js)

    data = eval_js('recordVideo()')
    binary = b64decode(data.split(',')[1])

    with open(filename, 'wb') as f:
        f.write(binary)

    return filename
```

Solution: To access web camera we need to go with python + java script code



Capturing video and converting it into gray scale image and printing its matrix

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# Load the video
video_path = "video.webm" # change to your video path
cap = cv2.VideoCapture(video_path)

# Read one frame
ret, frame = cap.read()

if ret:

    # Print the original frame matrix
    print("Original Frame Matrix:\n")
    print(frame)

    # Show the frame
    cv2_imshow(frame)

    # Convert frame to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Print grayscale matrix
    print("\nGrayscale Frame Matrix:\n")
    print(gray)

    # Show grayscale frame
    cv2_imshow(gray)

else:
    print("Error: Could not read frame from video.")

# Release video
cap.release()
```

SuperResolution

...



Grayscale Frame Matrix:

```
[[ 41  41  41 ... 168 168 168]
 [ 34  34  34 ... 168 168 168]
 [ 32  32  32 ... 168 168 168]
 ...
 [172 172 172 ...  29  29  29]
 [172 172 172 ...  29  29  29]
 [172 172 172 ...  29  29  29]]
```



Edge Detection using Canny

Edge Detection using Canny

```
from IPython.display import Javascript, display

display(Javascript("""
async function runEdgeDetection() {

    const div = document.createElement('div');
    const video = document.createElement('video');
    const canvas = document.createElement('canvas');
    const ctx = canvas.getContext('2d');

    div.appendChild(video);
    div.appendChild(canvas);
    document.body.appendChild(div);

    const stream = await navigator.mediaDevices.getUserMedia({video:true});
    video.srcObject = stream;
    await video.play();

    canvas.width = video.videoWidth;
    canvas.height = video.videoHeight;

    function processFrame() {
        ctx.drawImage(video,0,0,canvas.width,canvas.height);

        let frame = cv.imread(canvas);
        let gray = new cv.Mat();
        let edges = new cv.Mat();

        cv.cvtColor(frame, gray, cv.COLOR_RGBA2GRAY);
        cv.Canny(gray, edges, 100, 200);

        cv.imshow(canvas, edges);

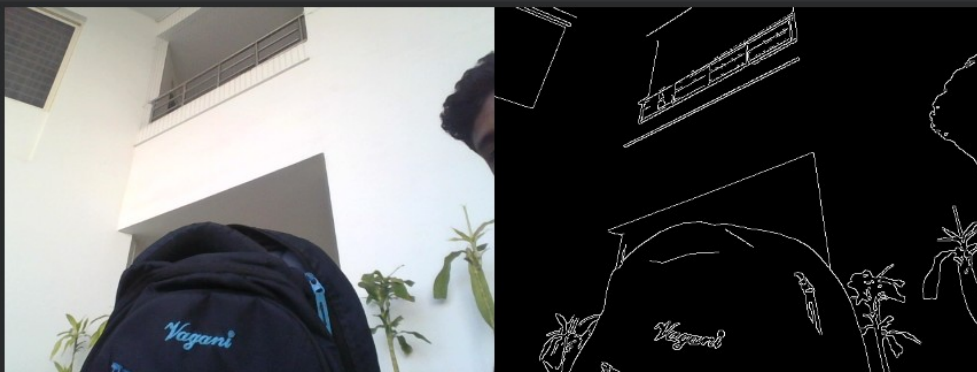
        frame.delete();
        gray.delete();
        edges.delete();

        requestAnimationFrame(processFrame);
    }

    processFrame();
})

var script = document.createElement('script');
script.src = "https://docs.opencv.org/4.x/opencv.js";
script.onload = runEdgeDetection;
document.body.appendChild(script);
"""))
```

```
document.body.appendChild(script);
"""))
```



```

import cv2
import numpy as np
from base64 import b64decode
from IPython.display import Javascript, display
from google.colab.output import eval_js
from google.colab.patches import cv2_imshow

def capture_image():

    js = Javascript('''
    async function takePhoto() {
        const video = document.createElement('video');
        const stream = await navigator.mediaDevices.getUserMedia({video:true});

        document.body.appendChild(video);
        video.srcObject = stream;
        await video.play();

        const canvas = document.createElement('canvas');
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;

        const ctx = canvas.getContext('2d');
        ctx.drawImage(video, 0, 0);

        stream.getTracks().forEach(track => track.stop());
        video.remove();

        return canvas.toDataURL('image/jpeg');
    }
    ''')

    display(js)

    data = eval_js('takePhoto()')
    binary = b64decode(data.split(',')[1])
    img = np.frombuffer(binary, np.uint8)
    frame = cv2.imdecode(img, cv2.IMREAD_COLOR)

    return frame

# Capture frame
frame = capture_image()

# Convert to grayscale
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Negative image
negative = 255 - gray

print("Original Matrix:\n")
print(gray)

print("\nNegative Matrix:\n")
print(negative)

cv2_imshow(gray)
cv2_imshow(negative)

```



```

*** Original Matrix:

[[204 204 204 ... 152 151 151]
 [204 204 205 ... 152 152 152]
 [203 205 205 ... 152 153 154]
 ...
 [ 18  19  21 ... 149 149 149]
 [ 22  21  22 ... 149 149 149]
 [ 26  24  23 ... 149 149 149]]

Negative Matrix:

[[ 51  51  51 ... 103 104 104]
 [ 51  51  50 ... 103 103 103]
 [ 52  50  50 ... 103 102 101]
 ...
 [237 236 234 ... 106 106 106]
 [233 234 233 ... 106 106 106]
 [229 231 232 ... 106 106 106]]

```

Detection of Edge using Mathematical formula

Detection of Edge Using Gradient Formula

```

import cv2
import numpy as np

# Read image
img = cv2.imread("image.jpg")

# Convert image to grayscale
# Edge detection works on intensity values
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Sobel kernel for detecting horizontal edges (Gx)
sobel_x = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
])

# Sobel kernel for detecting vertical edges (Gy)
sobel_y = np.array([
    [-1, -2, -1],
    [ 0,  0,  0],
    [ 1,  2,  1]
])

# Convolution operation to compute gradient in x direction
Gx = cv2.filter2D(gray, -1, sobel_x)

# Convolution operation to compute gradient in y direction
Gy = cv2.filter2D(gray, -1, sobel_y)

# Compute gradient magnitude using the formula
# G = sqrt(Gx^2 + Gy^2)
G = np.sqrt(Gx.astype(float)**2 + Gy.astype(float)**2)

# Normalize values to 0-255 range
G = np.uint8(G / G.max() * 255)

# Show results
cv2.imshow("Original Image", gray)
cv2.imshow("Gradient X (Gx)", Gx)
cv2.imshow("Gradient Y (Gy)", Gy)
cv2.imshow("Edge Magnitude", G)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

